

20 BASIC PYTHON PROGRAMS

Detailed explanations, worked examples, sample outputs, and important programming notes

Course	MAT337 - Data Science using Python
Experiment	Implementation of basic Python programs using conditional statements, loops, lists, dictionaries, and functions
Layout	Compact two-column reference designed for classroom study and lab-record preparation

How to use this guide: First read the purpose and code. Then follow the execution steps and worked example. Finally, run the sample input and test the important notes.

PART A: SIMPLE PROGRAMS

Program 1: Sum of First n Natural Numbers

Problem

Find the sum $1 + 2 + \dots + n$ for a nonnegative integer entered by the user.

Python Code

```
n = int(input("Enter n: "))
total = n * (n + 1) // 2
print("Sum:", total)
```

Main Idea

The program uses the formula $n(n + 1)/2$. This avoids a loop and therefore gives the result in constant time.

Step-by-Step Logic

- `input()` reads the value as text.
- `int()` converts the text to an integer.
- $n * (n + 1) // 2$ applies the formula.
- `//` gives an integer quotient.

Worked Example

For $n = 5$, the calculation is $5 \times 6 // 2 = 15$.

Sample Input/Output

```
Enter n: 5
Sum: 15
```

Important Notes

- The formula assumes $n \geq 0$.
- Time complexity: $O(1)$.

Program 2: Swap Two Numbers Without a Temporary Variable

Problem

Exchange the values stored in two variables.

Python Code

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
a, b = b, a
print("a =", a, "b =", b)
```

Main Idea

Python first evaluates the right side (b, a), then unpacks those two values into a and b .

Step-by-Step Logic

- Read the two integers.
- Create the pair (b, a).
- Assign the first value to a and the second to b .

Worked Example

If $a = 10$ and $b = 20$, the right side becomes $(20, 10)$; afterward $a = 20$ and $b = 10$.

Sample Input/Output

```
Enter a: 10
Enter b: 20
a = 20 b = 10
```

Important Notes

- This is called tuple unpacking.

- It works with numbers, strings, lists, and many other Python objects.

Program 3: Largest of Three Numbers

Problem

Determine the largest among three user-entered integers.

Python Code

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
c = int(input("Enter c: "))
print("Largest:", max(a, b, c))
```

Main Idea

The built-in `max()` function compares all arguments and returns the greatest value.

Step-by-Step Logic

- Read a, b , and c .
- Evaluate `max(a, b, c)`.
- Print the returned value.

Worked Example

For 25, 18, 30, Python compares the values and returns 30.

Sample Input/Output

```
Enter a: 25
Enter b: 18
Enter c: 30
Largest: 30
```

Important Notes

- Equal values are handled correctly.
- `max()` also works with lists and tuples.

Program 4: Reverse a String

Problem

Display the characters of a string in reverse order.

Python Code

```
s = input("Enter a string: ")
print("Reversed string:", s[::-1])
```

Main Idea

The slice `s[start:stop:step]` uses a step of -1 . Therefore, `s[::-1]` reads from the end to the beginning.

Step-by-Step Logic

- Read the complete line as a string.
- Create a reversed copy using `[::-1]`.
- Print the reversed copy.

Worked Example

For $s = \text{"Python"}$, the characters are read as n, o, h, t, y, P , producing `"nohtyP"`.

Sample Input/Output

```
Enter a string: Hello
Reversed string: olleH
```

Important Notes

- Strings are immutable; the original string is unchanged.
- Time complexity: $O(n)$.

Program 5: Count Digits in an Integer

Problem

Count the decimal digits in an integer without counting a minus sign.

Python Code

```
n = int(input("Enter an integer: "))
count = len(str(abs(n)))
print("Number of digits:", count)
```

Main Idea

`abs(n)` removes the sign, `str()` converts the number to characters, and `len()` counts them.

Step-by-Step Logic

- Read the integer.
- Apply `abs()`.
- Convert to a string.
- Count the string length.

Worked Example

For $n = -4567$: `abs(n) = 4567`, `str(...)` = `"4567"`, and `len(...)` = 4.

Sample Input/Output

```
Enter an integer: -4567
Number of digits: 4
```

Important Notes

- The number 0 correctly gives one digit.
- This method is simple and readable.

Program 6: Sum of Elements in a List

Problem

Read space-separated integers and calculate their total.

Python Code

```
lst = list(map(int, input("Enter list elements: ").split()))
print("Sum:", sum(lst))
```

Main Idea

split() separates the input, map(int, ...) converts each item, list() stores them, and sum() adds them.

Step-by-Step Logic

- Input: "10 20 30".
- split() gives ["10", "20", "30"].
- map(int, ...) gives integers.
- sum() returns 60.

Worked Example

For [10, 20, 30, 40], the total is 100.

Sample Input/Output

```
Enter list elements: 10 20 30 40
Sum: 100
```

Important Notes

- An empty input produces an empty list and sum([]) is 0.

Program 7: Average Marks of Students

Problem

Compute the arithmetic mean of a list of student marks.

Python Code

```
marks = list(map(float, input("Enter marks: ").split()))
average = sum(marks) / len(marks)
print("Average marks:", average)
```

Main Idea

The mean is sum of observations / number of observations. Floats are used so decimal marks are accepted.

Step-by-Step Logic

- Read and convert marks to floating-point values.
- Calculate sum(marks).
- Count values using len(marks).
- Divide the total by the count.

Worked Example

For 85, 90, 78, 92, 88, the sum is 433 and the average is $433/5 = 86.6$.

Sample Input/Output

```
Enter marks: 85 90 78 92 88
Average marks: 86.6
```

Important Notes

- At least one mark must be entered; otherwise division by zero occurs.
- Use print(f"{average:.2f}") for two decimal places.

PART B: PROGRAMS USING CONDITIONAL STATEMENTS

Program 8: Even or Odd Check

Problem

Classify an integer as even or odd.

Python Code

```
n = int(input("Enter an integer: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Main Idea

The modulo operator % returns the remainder. An even number leaves remainder 0 when divided by 2.

Step-by-Step Logic

- Evaluate $n \% 2$.
- If the remainder equals zero, execute the if block.
- Otherwise execute the else block.

Worked Example

For $n = 17$, $17 \% 2 = 1$; therefore the output is Odd.

Sample Input/Output

```
Enter an integer: 24
Even
```

Important Notes

- Zero is even.
- The test also works for negative integers.

Program 9: Palindrome Check

Problem

Check whether a string reads the same forward and backward.

Python Code

```
s = input("Enter a string: ")
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

Main Idea

The original string is compared with its reversed copy.

Step-by-Step Logic

- Read the string.
- Construct $s[::-1]$.
- Use == to compare both strings.
- Select the appropriate output.

Worked Example

"madam"[::-1] is "madam", so it is a palindrome. "hello"[::-1] is "olleh", so it is not.

Sample Input/Output

```
Enter a string: racecar
Palindrome
```

Important Notes

- The given program is case-sensitive and space-sensitive.
- For case-insensitive checking, compare s.lower() with its reverse.

Program 10: Armstrong Number Check

Problem

Check whether a nonnegative integer equals the sum of its digits raised to the number of digits.

Python Code

```
n = int(input("Enter a number: "))
digits = list(map(int, str(n)))
power = len(digits)
if sum(d ** power for d in digits) == n:
    print("Armstrong number")
else:
    print("Not an Armstrong number")
```

Main Idea

For a k-digit number, compute $d_1^{**k} + d_2^{**k} + \dots + d_k^{**k}$ and compare it with the original number.

Step-by-Step Logic

- Convert the number to a list of digits.
- Set power equal to the number of digits.
- Use a generator expression to compute each powered digit.
- Add the values and compare with n.

Worked Example

For 153: $1^{**3} + 5^{**3} + 3^{**3} = 1 + 125 + 27 = 153$.

Sample Input/Output

```
Enter a number: 153
Armstrong number
```

Important Notes

- Typical Armstrong numbers include 0, 1, 153, 370, 371, 407, 9474.
- The given code is intended for nonnegative integers.

PART C: PROGRAMS USING LOOPING CONSTRUCTS

Program 11: Factorial Using a Loop

Problem

Calculate $n! = 1 \times 2 \times \dots \times n$ using a loop.

Python Code

```
n = int(input("Enter a number: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("Factorial:", fact)
```

Main Idea

The variable fact is an accumulator. During every iteration it is multiplied by the next integer.

Step-by-Step Logic

- Initialize fact = 1.
- Generate integers from 1 through n.
- Update fact using fact *= i.
- Print the final product.

Worked Example

For $n = 5$: $1 \rightarrow 1 \rightarrow 2 \rightarrow 6 \rightarrow 24 \rightarrow 120$. Thus $5! = 120$.

Sample Input/Output

```
Enter a number: 5
Factorial: 120
```

Important Notes

- By definition, $0! = 1$, which this program returns.
- Negative inputs should be rejected in a robust version.

Program 12: First n Fibonacci Numbers**Problem**

Print the first n terms of the Fibonacci sequence.

Python Code

```
n = int(input("Enter n: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Main Idea

Each term is obtained from the previous two. The variables a and b store consecutive terms.

Step-by-Step Logic

- Start with a = 0 and b = 1.
- Print a.
- Simultaneously update to a = b and b = a + b.
- Repeat n times.

Worked Example

For n = 7, the sequence printed is 0 1 1 2 3 5 8.

Sample Input/Output

```
Enter n: 7
0 1 1 2 3 5 8
```

Important Notes

- The underscore _ means the loop counter itself is not needed.
- Time complexity: $O(n)$.

Program 13: Multiplication Table of a Number**Problem**

Print the first ten multiples of a user-entered number.

Python Code

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Main Idea

The loop variable i takes values from 1 to 10. Each row displays n x i.

Step-by-Step Logic

- Read n.
- Use range(1, 11); the ending value 11 is excluded.
- Multiply n by the current i.
- Print one table row per iteration.

Worked Example

For n = 6, the program prints 6 x 1 = 6 through 6 x 10 = 60.

Sample Input/Output

```
Enter a number: 3
3 x 1 = 3
...
3 x 10 = 30
```

Important Notes

- Change 11 to m + 1 to print up to any chosen multiplier m.

Program 14: Prime Numbers up to n**Problem**

Print every prime number from 2 through n.

Python Code

```
n = int(input("Enter n: "))
for num in range(2, n + 1):
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            break
    else:
        print(num, end=" ")
```

Main Idea

A number is prime when it has no divisor other than 1 and itself. It is sufficient to test divisors only up to $\sqrt{\text{num}}$.

Step-by-Step Logic

- The outer loop selects each candidate.
- The inner loop tests possible divisors.
- If a divisor is found, break stops the inner loop.
- The inner-loop else runs only when no break occurred, so the candidate is prime.

Worked Example

For num = 17, test divisors 2, 3, 4. None divides 17, so 17 is printed.

Sample Input/Output

```
Enter n: 20
2 3 5 7 11 13 17 19
```

Important Notes

- The else belongs to the inner for, not to the if.
- Testing only to the square root is much faster than testing to num - 1.

Program 15: Count Even Numbers in a List**Problem**

Count how many list elements are even.

Python Code

```
lst = list(map(int, input("Enter list elements: ").split()))
count = sum(1 for x in lst if x % 2 == 0)
print("Even count:", count)
```

Main Idea

The generator produces 1 for each element satisfying $x \% 2 == 0$. Summing those ones gives the count.

Step-by-Step Logic

- Read the integer list.
- Visit each value x.
- Keep only values passing the even test.
- Generate one for each match and add them.

Worked Example

For [4, 7, 10, 13, 18], the matching values are 4, 10, 18; the result is 3.

Sample Input/Output

```
Enter list elements: 4 7 10 13 18
Even count: 3
```

Important Notes

- This is a compact alternative to using a loop and manually incrementing a counter.

Program 16: Count Vowels in a String**Problem**

Count the uppercase and lowercase English vowels in a string.

Python Code

```
s = input("Enter a string: ")
vowels = "aeiouAEIOU"
count = sum(1 for ch in s if ch in vowels)
print("Vowel count:", count)
```

Main Idea

Each character is tested for membership in the string "aeiouAEIOU".

Step-by-Step Logic

- Read the string.
- Visit one character at a time.
- Check ch in vowels.
- Generate and sum one for each vowel.

Worked Example

For "Data Science", the vowels are a, a, i, e, e; therefore the count is 5.

Sample Input/Output

```
Enter a string: Data Science
Vowel count: 5
```

Important Notes

- Spaces, digits, and consonants are ignored.
- An alternative is if ch.lower() in "aeiou".

Program 17: Remove Duplicate Elements from a List**Problem**

Create a list containing only distinct values.

Python Code

```
lst = list(map(int, input("Enter list elements: ").split()))
unique = list(set(lst))
print("Without duplicates:", unique)
```

Main Idea

A set stores each value only once. Converting the set back to a list gives a duplicate-free list.

Step-by-Step Logic

- Read the list.
- Convert it to a set; repeated values collapse into one.
- Convert the set back to a list.
- Print the result.

Worked Example

For [2, 4, 2, 7, 4, 9], the distinct values are 2, 4, 7, 9.

Sample Input/Output

Enter list elements: 2 4 2 7 4 9
Without duplicates: [2, 4, 7, 9]

Important Notes

- A set does not guarantee the original order.
- To preserve order, use `list(dict.fromkeys(lst))`.

Program 18: Word Frequency Using a Dictionary**Problem**

Count how many times each word appears in a sentence.

Python Code

```
sentence = input("Enter a sentence: ")
words = sentence.split()
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

Main Idea

A dictionary maps every word to its count. `freq.get(word, 0)` returns the current count or zero when the word is new.

Step-by-Step Logic

- Split the sentence into words.
- Start with an empty dictionary.
- For each word, read its old count.
- Add one and store the updated count.

Worked Example

For "data is useful and data is powerful", data and is each have frequency 2.

Sample Input/Output

Enter a sentence: red blue red green
{'red': 2, 'blue': 1, 'green': 1}

Important Notes

- The basic version is case-sensitive and treats punctuation as part of a word.
- Use cleaning and `lower()` for normalized counting.

PART D: PROGRAMS USING USER-DEFINED FUNCTIONS**Program 19: Function to Add Two Numbers****Problem**

Define and call a reusable function that returns the sum of two values.

Python Code

```
def add(a, b):
    return a + b

x = int(input("Enter x: "))
y = int(input("Enter y: "))
print("Sum:", add(x, y))
```

Main Idea

`def` creates a function. The parameters `a` and `b` receive arguments, and `return` sends the result back to the caller.

Step-by-Step Logic

- Define `add(a, b)`.
- Read `x` and `y`.
- Call `add(x, y)`.
- Print the returned value.

Worked Example

When `x = 12` and `y = 8`, the function receives `a = 12`, `b = 8`, and returns 20.

Sample Input/Output

Enter x: 12
Enter y: 8
Sum: 20

Important Notes

- A function reduces repetition and makes code easier to test.
- The same function also works with floats if the inputs are converted using `float()`.

Program 20: Function to Return the Maximum Element in a List**Problem**

Define a function that returns the largest value in a list.

Python Code

```
def max_element(lst):
    return max(lst)

lst = list(map(int, input("Enter list elements: ").split()))
print("Maximum:", max_element(lst))
```

Main Idea

The function receives a list through its parameter and delegates the comparison to Python's built-in `max()`.

Step-by-Step Logic

- Define `max_element(lst)`.
- Return `max(lst)`.
- Read the list from the user.
- Pass the list to the function and print the result.

Worked Example

For [12, 45, 7, 31], `max(lst)` returns 45.

Sample Input/Output

Enter list elements: 12 45 7 31
Maximum: 45

Important Notes

- The list must contain at least one element.
- The function separates the operation from input/output logic.

QUICK REFERENCE: CORE PYTHON IDEAS USED

Concept	Python form
Input conversion	<code>int(input(...))</code> , <code>float(input(...))</code>
String splitting	<code>input().split()</code>
List conversion	<code>list(map(int, ...))</code>
Decision making	<code>if</code> , <code>elif</code> , <code>else</code>
Iteration	<code>for</code> , <code>range()</code>
Useful built-ins	<code>sum()</code> , <code>len()</code> , <code>max()</code> , <code>abs()</code>
Collections	lists, sets, and dictionaries
Functions	<code>def</code> , parameters, arguments, and <code>return</code>

Practice recommendation: Run each program with normal, boundary, and unusual inputs. Then rewrite at least one compact solution using a loop and one loop-based solution using a built-in function.

Common Errors to Avoid

- Forgetting that `input()` returns a string.
- Using incorrect indentation inside `if`, `for`, or function blocks.
- Forgetting that the stop value in `range()` is excluded.
- Calling `max()` or dividing by `len()` when a list is empty.
- Assuming `set()` preserves the original list order.

Minimum Testing Checklist

- Test an ordinary positive input.
- Test zero whenever it is meaningful.
- Test a negative value when the program accepts integers.
- Test repeated values for list and frequency programs.
- Test uppercase letters, spaces, and punctuation for string programs.

Learning Outcome

After completing these programs, students should be able to read input, perform type conversion, use expressions and built-in functions, construct decisions and loops, process strings and lists, use dictionaries and sets, and define simple reusable functions.